```
package com.kuang.single.my;

public class StaticSingle {
    private StaticSingle() {
    };
    public static StaticInner getInstance() {
        return new StaticInner();
    }
    static class StaticInner{

    }
}
```

我这样写为啥测试时不符合单例呢?

为什么不符合单例?

核心问题就一行:

```java
public static StaticInner getInstance() {
    return new StaticInner();   // ❌ 每次都 new 一个新的
}
```

问题分析

你的代码有两个致命问题:

1. `getInstance()` 每次调用都 `new`

```java
StaticInner a = StaticSingle.getInstance();
StaticInner b = StaticSingle.getInstance();
System.out.println(a == b); // false，每次都创建新对象
```

单例的核心要求是**全局只有一个实例**，而你每次调用都返回一个新的 `StaticInner`，所以 `a != b`，自然不是单例。

2. `StaticInner` 没有私有构造，且返回类型不是 `StaticSingle`

- `StaticInner` 是包级访问权限的静态内部类，没有私有构造，外部还能直接 `new StaticSingle.StaticInner()`，根本不受控。
- 单例通常返回的是**单例类自身**，而你的 `getInstance()` 返回的是内部类，逻辑上也不对。

正确的静态内部类单例写法

静态内部类单例（Holder 模式）的标准写法是：

```
java

public class StaticSingle {

    private StaticSingle() {
    }

    // 静态内部类，只有在第一次被引用时才会初始化（类加载机制保证线程安全）
    private static class StaticInner {
        private static final StaticSingle INSTANCE = new StaticSingle();
    }

    public static StaticSingle getInstance() {
        return StaticInner.INSTANCE;    // ✅ 每次都返回同一个实例
    }
}
```

要点：

关键点	说明
`StaticSingle` 构造私有	防止外部 `new StaticSingle()`
`INSTANCE` 是 `static final`	只初始化一次
`getInstance()` 返回 `StaticInner.INSTANCE`	不 new，只是取引用
内部类用 `private static`	外部无法直接访问内部类

测试验证

```
java
```

```
public class Test {
    public static void main(String[] args) {
        StaticSingle a = StaticSingle.getInstance();
        StaticSingle b = StaticSingle.getInstance();
        System.out.println(a == b); // true ✅
    }
}
```

补充说明：Holder 模式为什么线程安全

JVM 在**类初始化阶段**会加锁，保证 `<clinit>` 只执行一次，所以 `INSTANCE` 只会被创建一次，且是**懒加载**（`StaticInner` 在第一次调用 `getInstance()` 时才被加载）。这就是它比饿汉式更优、比双重检查锁更简洁的原因。

一句话总结：**你的 `getInstance()` 里写了 `new`，每次调用都会创建新对象，当然不是单例。**

我这个静态内部类不是只会有一个吗？为啥我这里new的时候还会有两个

关键区别：类只有一个 vs 对象可以有很多个

你把**"类"**和**"类的实例"**混淆了。这是理解这个问题的核心。

你现在的代码实际发生了什么

```
java

public static StaticInner getInstance() {
    return new StaticInner();   // 每次调用都执行 new
}
```

`StaticInner` **这个类**确实只有一个（由类加载器加载，JVM 里就只有一份 `Class` 对象）。但是：

> **类只有一个 ≠ 类的对象只有一个**

每次执行 `new StaticInner()`，JVM 都会在堆上**新分配一块内存**，创建一个全新的对象。

```
java

StaticInner a = StaticSingle.getInstance(); // 堆上创建对象①
StaticInner b = StaticSingle.getInstance(); // 堆上创建对象②
```

```
System.out.println(a == b); // false，两个不同的对象
```

用图表示：

```
text

StaticInner 类（唯一）
      │
      ├── new 出来 → 对象① (a 指向它)
      └── new 出来 → 对象② (b 指向它)
```

`new` 是"创建对象"的操作，调用一次就造一个。类只有一个，但它可以被 `new` 无数次。

单例到底要求什么？

单例要求的是**对象**只有一个，而不是类只有一个：

```
java

// 错误理解
"StaticInner 类只加载一次，所以是单例"  ❌

// 正确理解
"StaticInner 类里有个 static 字段只存了一个对象，所有地方都拿这一个"  ✅
```

Holder 模式为什么能保证对象只有一个

看正确写法：

```
java

private static class StaticInner {
    private static final StaticSingle INSTANCE = new StaticSingle();
}

public static StaticSingle getInstance() {
    return StaticInner.INSTANCE;   // 注意：这里没有 new！
}
```

拆解一下：

1. `private static final StaticSingle INSTANCE = new StaticSingle();`
   这句 `new` 只会在**类初始化时执行一次。** `StaticInner` 类被 JVM 初始化时，执行 `<clinit>` 静态初始化块，创建一个对象，存到静态字段 `INSTANCE` 里。之后这个字段就一直指向这**同一个对象。**
2. `return StaticInner.INSTANCE;`
   每次调用 `getInstance()`，只是**读取**这个静态字段的引用，返回给调用方。全程**没有再** `new` 。

所以：

```
java

StaticSingle a = StaticSingle.getInstance(); // 返回 INSTANCE，指向对象①
StaticSingle b = StaticSingle.getInstance(); // 还是返回 INSTANCE，还是对象①
System.out.println(a == b); // true ✅
```

图示：

```
text

StaticInner 类（唯一）
      |
      └── static final INSTANCE ──→ 对象①（唯一，创建一次）
                                          ▲
                                          |
                          getInstance() 每次都返回这个引用
```

对比总结

写法	`new` 执行次数	对象数量
你的写法：`return new StaticInner()`	每次调用都执行	调用几次就几个
Holder：`return StaticInner.INSTANCE`	类初始化时执行 1 次	永远 1 个

一句话点破

> **类只有一个，是"模板"；** `new` **一次就照着模板造一个"产品"。单例要的是产品只有一个，所以** `new` **只能执行一次，之后都用同一个引用。**

你原来的代码里 `getInstance()` 每次都 `new`，相当于每次调用都"照模板生产一个新产品"，当然就不是单例了。Holder 模式把 `new` 挪到静态字段初始化里（只执行一次），`getInstance()` 只负责"把那个唯一产品递给你"，这才是单例。

> 按理说每一个类都是唯一的，为啥这里要求是静态内部类呢？我直接一个内部类不行吗？

为什么必须是 `static` 内部类？普通内部类不行吗？

先纠正一个容易混淆的点：**"类唯一"和"用 static 内部类"是两码事。**

- "类唯一"说的是：`StaticInner.class` 这个 Class 对象在 JVM 里只有一份 —— 这是**所有类**都成立的，跟是否 static 无关。
- "为什么要求静态内部类"说的是：**内部类要能独立于外部类实例被加载、被访问。**这跟 static 有关系。

核心区别：非静态内部类"寄生"在外部类实例上

普通（非静态）内部类有一个隐藏特性：

> 非静态内部类的每个实例，**必须绑定一个外部类实例。**而且非静态内部类**不能有自己的 static 成员**（Java 16 之前）。

也就是说，如果写成非静态内部类：

```
java

public class StaticSingle {
    private StaticSingle() {}

    // 非静态内部类
    class StaticInner {
        // ❌ 编译错误（Java 16 之前）：
        // 非静态内部类不能声明 static 成员
        private static final StaticSingle INSTANCE = new StaticSingle();
    }

    public static StaticSingle getInstance() {
        // ❌ 也没法直接 new StaticInner()：
        // 非静态内部类需要外部类实例
        // return StaticInner.INSTANCE;
```

```
    }
}
```

问题立刻出现：

问题 1：非静态内部类不能有 static 字段

单例的核心就是"用一个 static 字段保存唯一实例"。而非静态内部类**不允许**有 static 成员（静态常量 `static final` 基本类型/字符串除外，但这里要存对象引用，不行）。

所以你根本没法在里面写 `private static final StaticSingle INSTANCE = ...`，Holder 模式的前提就没了。

问题 2：`getInstance()` 是 static 的，没法访问非静态内部类

`getInstance()` 是静态方法，没有 `this`（外部类实例）。而非静态内部类要创建实例，必须先有外部类实例：

```java
// 非静态内部类的创建方式，前提是有一个外部类对象 outer
StaticSingle outer = new StaticSingle();
StaticSingle.StaticInner inner = outer.new StaticInner();
```

但 `getInstance()` 是 static 的，手里没有 `outer`，压根 `new` 不出来。死循环：要拿单例得先有外部实例，要外部实例又违背单例初衷。

问题 3：非静态内部类会持有外部类引用，还有"隐式加载"依赖

非静态内部类编译后是 `Outer$Inner`，它的构造器签名里隐含了外部类引用，跟外部类实例强绑定。这跟"独立、懒加载、只初始化一次"的 Holder 设计完全冲突。

静态内部类为什么正好合适

```java
public class StaticSingle {
    private StaticSingle() {}

    private static class StaticInner {                    // ✅ static
        private static final StaticSingle INSTANCE = new StaticSingle();
```

```
    }

    public static StaticSingle getInstance() {
        return StaticInner.INSTANCE;                    // ✅ 静态方法能直接访问
    }
}
```

`static` 带来了三个关键能力：

能力	说明
可以有自己的 static 成员	能写 `static final INSTANCE`，这是 Holder 的载体
可以脱离外部类实例独立存在	不需要 `outer.new`，`getInstance()` 直接访问 `StaticInner.INSTANCE`
独立的类初始化时机	`StaticInner` 只在**第一次被引用**时才触发类初始化，实现懒加载；JVM 保证 `<clinit>` 只执行一次，天然线程安全

懒加载这一点特别关键：`StaticSingle` 类加载时，`StaticInner` **不会**被加载；只有第一次调用 `getInstance()`、真正去读 `StaticInner.INSTANCE` 时，JVM 才初始化 `StaticInner`，这时才执行 `new StaticSingle()`。这就是"用到才创建"。

如果非要不用 static?

那你就得回到更笨的写法，比如饿汉式（在外部类自己的 static 字段里创建）：

```
java

public class StaticSingle {
    private StaticSingle() {}
    private static final StaticSingle INSTANCE = new StaticSingle(); // 外部类加载时就创建，非懒加载
    public static StaticSingle getInstance() { return INSTANCE; }
}
```

这是饿汉式，能用，但**不是懒加载**：类一加载就创建对象，不管你用不用。Holder 模式的价值就是用 static 内部类把"懒加载 + 线程安全 + 简洁"三者同时拿到。

一句话总结

> **"类唯一"是所有类的共性，不是选 static 的理由。选 `static` 内部类，是因为只有静态内部类才能：① 拥有 `static final INSTANCE` 字段来存唯一对象；② 被静态方法 `getInstance()` 直接访问；③ 延迟到首次使用时才初始化，借助 JVM 类加载机制天然保证线程安全。非静态内部类寄生在外部实例上、不能有 static 成员，根本承载不了单例。**

所以不是"每个类都唯一所以随便用内部类"，而是"**单例需要一个能独立、懒加载、持有唯一 static 实例的容器，静态内部类恰好是那个容器**"。

所以这个唯一性是由static来保证的？用静态内部类是为了懒加载，那如何保证线程安全的呢？

唯一性由谁保证？线程安全又由谁保证？

你的理解方向对，但要拆成两件事看，别混在一起：

- **唯一性**：由 `static final` 字段 + "只 `new` 一次" 保证（`static` 让字段属于类、全类共享；`final` + 只赋值一次保证不被覆盖）。
- **懒加载**：由"静态内部类首次被引用才初始化"保证。
- **线程安全**：**不是 `static` 本身直接保证的**，而是 **JVM 的类初始化机制（`<clinit>` 加锁）** 保证的。

下面重点说线程安全。

线程安全的真正来源：JVM 类初始化锁

先看 Holder 模式的关键代码：

```
java

public class StaticSingle {
    private StaticSingle() {}

    private static class StaticInner {
        private static final StaticSingle INSTANCE = new StaticSingle(); // ← 这句在 <clinit> 里
    }

    public static StaticSingle getInstance() {
        return StaticInner.INSTANCE;
    }
}
```

`INSTANCE = new StaticSingle()` 这句，编译后属于 `StaticInner` 类的**静态初始化块** `<clinit>`。而 JVM 规范对 `<clinit>` 有明确规定：

> **JVM 保证一个类的 `<clinit>` 方法在多线程环境下被正确地加锁、同步。如果多个线程同时去初始化一个类，只有一个线程会执行 `<clinit>`，其他线程阻塞等待，直到 `<clinit>` 执行完毕。**

并发场景推演

假设线程 A 和线程 B 同时第一次调用 `getInstance()`：

```
text

线程A: getInstance() → 读 StaticInner.INSTANCE
线程B: getInstance() → 读 StaticInner.INSTANCE
                │
                ▼
     两者都触发 StaticInner 类初始化
                │
                ▼
   JVM 给 StaticInner 的初始化加锁
                │
        ┌───────┴────────┐
        ▼                ▼
   线程A 抢到锁        线程B 阻塞等待
        │                │
   执行 <clinit>         │
   new StaticSingle()    │
   赋值给 INSTANCE       │
        │                │
   释放锁 ──────────────→│
                         ▼
                  线程B 拿到锁，
                  发现已初始化，直接跳过
                         │
                         ▼
              两个线程都读到同一个 INSTANCE
```

所以：

- `new StaticSingle()` **只会执行一次**（在 A 线程的 `<clinit>` 里）。
- B 线程不会重复执行，也不会读到"半初始化"的对象（JVM 的锁 + happens-before 保证 A 的写入对 B 可见）。

这就是线程安全的根本来源：JVM 对类初始化的同步保证。 不是 `static` 关键字本身，`static` 只是让字段属于类；真正防并发的是 `<clinit>` 的加锁机制。

对比：为什么双重检查锁要自己写 volatile + synchronized

反过来说就更清楚了。DCL（双重检查锁）单例：

```
java

public class DclSingle {
    private static volatile DclSingle instance;   // 必须 volatile
    public static DclSingle getInstance() {
        if (instance == null) {                    // 第一次检查
            synchronized (DclSingle.class) {
                if (instance == null) {            // 第二次检查
                    instance = new DclSingle();    // 可能重排序!
                }
            }
        }
        return instance;
    }
}
```

DCL 里，**JVM 不会替你保证线程安全**，得自己解决两个问题：

问题	解决手段
多线程同时进入 `if (instance == null)`	手动 `synchronized` + 二次检查
`new` 的指令重排序导致读到未初始化对象	手动 `volatile` 禁止重排序

而 Holder 模式**一行 `volatile`、一个 `synchronized` 都不用写**，因为把"创建实例"这件事交给了 JVM 的类初始化，JVM 已经帮你加锁并处理了内存可见性。

三个"保证"分别归谁

目标	保证者	机制
实例唯一	`static final` 字段 + 只 `new` 一次	字段属于类、全类共享，只赋值一次
懒加载	静态内部类	`StaticInner` 首次被引用才触发类初始化

目标	保证者	机制
线程安全	**JVM**	类 `<clinit>` 初始化加锁 + happens-before

一个容易踩的补充点

严格说，Holder 模式的线程安全依赖"**类初始化只会发生一次**"这个 JVM 保证。但有一个边界情况要知道：

- **反射/序列化**仍可能破坏单例（反射可以调私有构造、序列化反序列化会 new 新对象），这跟线程安全无关，是单例的通用破绽，需要额外写 `readResolve()`、或在构造器里加防反射判断。
- 类加载器层面，如果同一个类被**不同的类加载器**加载，会有多个 `Class` 对象，各自初始化各自的 `INSTANCE` —— 这是"类唯一"在类加载器维度上的例外，一般业务代码不会遇到。

一句话总结

> **唯一性靠 `static final` 字段只被 `new` 一次；懒加载靠静态内部类"首次引用才初始化"；线程安全靠 JVM 对类 `<clinit>` 的加锁同步 —— 这就是 Holder 模式不用写 `volatile`、不用写 `synchronized` 却线程安全的原因。`static` 是"让字段归类共享"的手段，不是线程安全的直接原因。**